DevOps Experts Project

Full Reconstruction Guide

Phase 1 - Flask App & Docker

Phase 2 - Kubernetes with Minikube

Phase 3 - Helm Chart & Jenkins CI/CD

Every command, every issue, every fix - in order

Docker Hub: uzumaki420/devops-experts-project:latest

GitHub: github.com/byakugen-sketch/devops-experts-project

June 2026

Phase 1 - Flask Application & Docker

Phase 1 builds a minimal Python Flask web application, containerises it with Docker, runs it with Docker Compose, and publishes the image to Docker Hub. The app exposes three HTTP endpoints that are used by Kubernetes health probes in Phase 2.

Prerequisites

- Docker Desktop installed and running
- Python 3.11+ (for running without Docker)
- Docker Hub account with an access token

Application Code

Phase 1/app.py

```
from flask import Flask, jsonify
import os

app = Flask(__name__)

@app.route("/")
def home():
    return jsonify({"message": "Hello, Doron!"}), 200

@app.route("/health")
def health():
    return jsonify({"status": "healthy"}), 200

@app.route("/version")
def version():
    return jsonify({"version": os.environ.get("APP_VERSION", "1.0.0")}), 200

if __name__ == "__main__":
    debug = os.environ.get("FLASK_DEBUG", "false").lower() == "true"
    app.run(host="0.0.0.0", port=5000, debug=debug)
```

The app binds to 0.0.0.0 so it is reachable from outside the container. Debug mode is driven by the FLASK_DEBUG environment variable so it can be toggled without code changes.

Phase 1/requirements.txt

```
flask>=3.0.0
pytest>=8.0.0
```

Phase 1/Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

The slim base image keeps the final image small. Dependencies are installed before copying the rest of the code so Docker can cache the pip install layer and avoid re-running it on every code change.

Phase 1/.dockerignore

```
__pycache__
*.pyc
*.pyo
.env
.git
tests/
```

Prevents test files, cache, and secrets from leaking into the image.

Phase 1/docker-compose.yml

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - .:/app
    environment:
      - FLASK_DEBUG=false
```

Tests

Phase 1/tests/test_app.py

```
import pytest
from app import app

@pytest.fixture
def client():
    app.config['TESTING'] = True
```

```
    with app.test_client() as client:
        yield client

def test_home(client):
    response = client.get('/')
    assert response.status_code == 200
    assert response.json == {'message': 'Hello, Doron!'}

def test_health(client):
    response = client.get('/health')
    assert response.status_code == 200
    assert response.json == {'status': 'healthy'}

def test_version(client):
    response = client.get('/version')
    assert response.status_code == 200
    assert 'version' in response.json
```

Step-by-Step Commands

Option A - Run without Docker

```
cd "Phase 1"
pip install -r requirements.txt
python app.py
# Visit: http://localhost:5000
```

Option B - Build and run with Docker

```
cd "Phase 1"
docker build -t devops-experts-project .
docker run -p 5000:5000 devops-experts-project
# Visit: http://localhost:5000
```

Option C - Docker Compose

```
cd "Phase 1"
docker-compose up           # standard run
FLASK_DEBUG=true docker-compose up   # with debug mode
docker-compose down         # tear down
```

Run tests

```
cd "Phase 1"
pytest tests/ -v
```

Push image to Docker Hub

```
docker login
docker build -t uzumaki420/devops-experts-project:latest "Phase 1"
docker push uzumaki420/devops-experts-project:latest

# Pull and run directly from Docker Hub:
docker run -p 5000:5000 uzumaki420/devops-experts-project:latest
```

Issues Encountered & Fixes

[!] Issue: Home route returned plain text instead of JSON

The original home route returned the plain string 'Hello, Doron!' while the health and version routes returned JSON. This caused the healthcheck CronJob in Phase 2 to fail when comparing responses, and made the API inconsistent.

[OK] Fix Applied

Changed the home route to use jsonify({'message': 'Hello, Doron!'}) and updated the test to assert response.json == {'message': 'Hello, Doron!'} instead of response.data == b'Hello, Doron!'. This is also why the CronJob healthcheck later uses grep instead of an exact equality check - grep matches the string inside the JSON wrapper.

Phase 2 - Kubernetes with Minikube

Phase 2 deploys the Docker image from Phase 1 onto a local Kubernetes cluster running inside Docker via Minikube. It introduces Deployments, Services, Horizontal Pod Autoscaling, ConfigMaps, Secrets, and CronJobs.

Prerequisites

- Docker Desktop running
- Minikube: brew install minikube (or download from minikube.sigs.k8s.io)
- kubectl: brew install kubectl

Step 1 - Start the Cluster

```
minikube start --driver=docker

# Enable the Metrics Server (required for HPA)
minikube addons enable metrics-server

# Verify
minikube status
kubectl get nodes
# Expected: one node named 'minikube' with status Ready
```

The --driver=docker flag runs the Minikube node inside a Docker container rather than a VM. The Metrics Server addon exposes CPU and memory usage per pod - without it the HPA cannot read utilisation and will show <unknown> as the current CPU value.

Step 2 - Create the Secret

secret.yaml is intentionally excluded from git. Generate a base64-encoded key and create the file before applying any manifests.

```
# Generate a random base64-encoded secret key
python3 -c "import secrets, base64; print(base64.b64encode(secrets.token_hex(32).encode()).decode())"

# Create Phase 2/secret.yaml from the example
cp "Phase 2/secret.yaml.example" "Phase 2/secret.yaml"
# Edit the file and replace <base64-encoded-value> with the generated value
```

Phase 2/secret.yaml.example (template)

```
apiVersion: v1
kind: Secret
metadata:
  name: flask-secret
```

```
type: Opaque
data:
  SECRET_KEY: <base64-encoded-value>
```

NEVER commit the real secret.yaml. It is gitignored for a reason - secrets committed to git live in history permanently and can be extracted even after deletion. In production, use HashiCorp Vault, AWS Secrets Manager, or Kubernetes Sealed Secrets.

Step 3 - Apply All Manifests

Apply in this exact order so each resource exists before the ones that reference it:

```
kubectl apply -f "Phase 2/configmap.yaml"
kubectl apply -f "Phase 2/secret.yaml"
kubectl apply -f "Phase 2/deployment.yaml"
kubectl apply -f "Phase 2/service.yaml"
kubectl apply -f "Phase 2/hpa.yaml"
kubectl apply -f "Phase 2/cronjob-healthcheck.yaml"
kubectl apply -f "Phase 2/cronjob-cleanup.yaml"

# Verify everything is up
kubectl get pods
kubectl get services
kubectl get hpa
kubectl get cronjobs
```

Manifest Details

configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: flask-config
data:
  FLASK_DEBUG: "false"
  APP_NAME: "devops-experts-project"
  APP_VERSION: "1.0.0"
```

Non-sensitive env vars injected into every pod via envFrom in the Deployment.

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
spec:
  replicas: 2
```

```
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
        - name: flask-app
          image: uzumaki420/devops-experts-project:latest
          ports:
            - containerPort: 5000
          envFrom:
            - configMapRef:
                name: flask-config
          env:
            - name: SECRET_KEY
              valueFrom:
                secretKeyRef:
                  name: flask-secret
                  key: SECRET_KEY
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 10
            periodSeconds: 10
            failureThreshold: 3
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 5
            failureThreshold: 3
          resources:
            requests:
              cpu: "100m"
            limits:
              cpu: "200m"
```

Liveness probe: Kubernetes restarts the container after 3 consecutive failures on /health. Readiness probe: removes the pod from the Service load balancer without restarting it. CPU requests/limits are required for the HPA to calculate utilisation percentages.

service.yaml

```
apiVersion: v1
```

```
kind: Service
metadata:
  name: flask-service
spec:
  type: LoadBalancer
  selector:
    app: flask-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
```

LoadBalancer type exposes the app on port 80. On Mac with the Docker driver, an external IP is only assigned once minikube tunnel is running.

hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: flask-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: flask-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

Scales pods between 2 and 10 when average CPU across all pods exceeds 50% of their request.

cronjob-healthcheck.yaml

```
schedule: "*/1 * * * *"   # every minute
# Runs wget against flask-service:80, greps for 'Hello, Doron!'
# Exits with code 1 on failure so Kubernetes marks the job as failed
```

cronjob-cleanup.yaml

```
schedule: "0 * * * *"   # every hour
# Runs rm -rf /tmp/* inside a busybox container
```

Step 4 - Access the App

On macOS with the Docker driver, LoadBalancer services need a tunnel:

```
# Run in a separate terminal - keep it open
minikube tunnel

# App is now available at:
curl http://127.0.0.1:80
curl http://127.0.0.1:80/health
curl http://127.0.0.1:80/version
```

Verification Commands

```
# Pod health and probe events
kubectl describe pod <pod-name>

# HPA status and current CPU
kubectl get hpa
kubectl top pods

# Watch pods scale in real time
kubectl get pods -w

# CronJob history and logs
kubectl get jobs
kubectl logs -l job-name=<job-name>
```

Tear Down

```
kubectl delete -f "Phase 2/cronjob-healthcheck.yaml"
kubectl delete -f "Phase 2/cronjob-cleanup.yaml"
kubectl delete -f "Phase 2/hpa.yaml"
kubectl delete -f "Phase 2/service.yaml"
kubectl delete -f "Phase 2/deployment.yaml"
kubectl delete -f "Phase 2/configmap.yaml"
kubectl delete -f "Phase 2/secret.yaml"
minikube stop
```

Issues Encountered & Fixes

[!] Issue: HPA shows <unknown> for CPU

After applying hpa.yaml, kubectl get hpa shows TARGETS as <unknown>/50%. The HPA cannot read CPU metrics because the Metrics Server addon was not enabled.

[OK] Fix Applied

Run: minikube addons enable metrics-server

Wait ~60 seconds for the metrics pipeline to warm up, then check again with kubectl top pods. The HPA will start showing real percentages once the Metrics Server is collecting data.

[!] Issue: Service stays in <pending> external IP

kubectl get services shows the EXTERNAL-IP as <pending> indefinitely on macOS with the Docker driver.

[OK] Fix Applied

Run minikube tunnel in a separate terminal. This creates a network route from the host to the Minikube cluster and assigns 127.0.0.1 as the external IP for LoadBalancer services. The tunnel must stay running; closing it removes the route.

[!] Issue: Healthcheck CronJob failing after home route changed to JSON

The CronJob used an exact string comparison: ["$RESPONSE" = "Hello, Doron!"]. After the home route was changed to return JSON, the response became {"message": "Hello, Doron!"} and the exact comparison failed, causing the job to exit 1 on every run.

[OK] Fix Applied

Replaced the exact comparison with a grep:

if echo "$RESPONSE" | grep -q "Hello, Doron!"; then

grep -q checks whether the string appears anywhere in the response, so it matches regardless of the JSON wrapper. The same fix was applied in the Helm template in Phase 3.

Phase 3 - Helm Chart & Jenkins CI/CD

Phase 3 packages all Phase 2 Kubernetes manifests into a Helm chart so the entire deployment can be installed, upgraded, and uninstalled with a single command. A Jenkins pipeline running inside Docker automates build, test, push, and deploy whenever code is pushed to GitHub.

Prerequisites

- Docker Desktop running
- Minikube running: minikube start --driver=docker
- minikube tunnel running in a separate terminal
- Helm: brew install helm
- Docker Hub account and access token
- GitHub repository with the project code

Helm Chart Structure

```
Phase 3/
+-- Chart.yaml              # chart metadata
+-- values.yaml             # all configurable defaults
+-- templates/
|   +-- deployment.yaml     # Helm-templated Deployment
|   +-- service.yaml
|   +-- hpa.yaml
|   +-- configmap.yaml
|   +-- secret.yaml         # uses b64enc filter
|   +-- cronjob-healthcheck.yaml
|   +-- cronjob-cleanup.yaml
+-- jenkins/
    +-- Dockerfile           # Jenkins + Docker + kubectl + Helm
    +-- docker-compose.yml   # mounts Docker socket and kubeconfig
    +-- kubeconfig           # modified for host.docker.internal
```

Chart.yaml

```
apiVersion: v2
name: flask-app
description: Flask hello world app deployed on Kubernetes
type: application
version: 1.0.0
appVersion: "latest"
```

values.yaml

```
image:
  repository: uzumaki420/devops-experts-project
  tag: latest

replicaCount: 2

service:
  type: LoadBalancer
  port: 80
  targetPort: 5000

hpa:
  minReplicas: 2
  maxReplicas: 10
  cpuUtilization: 50

resources:
  requests:
    cpu: "100m"
  limits:
    cpu: "200m"

config:
  flaskDebug: "false"
  appName: "devops-experts-project"
  appVersion: "1.0.0"

secret:
  secretKey: ""

probes:
  liveness:
    initialDelaySeconds: 10
    periodSeconds: 10
    failureThreshold: 3
  readiness:
    initialDelaySeconds: 5
    periodSeconds: 5
    failureThreshold: 3

cronjobs:
  healthcheck:
    schedule: "*/1 * * * *"
  cleanup:
    schedule: "0 * * * *"
```

The secret.secretKey is intentionally blank in values.yaml. It must be passed at install/upgrade time via --set so it never appears in source control.

Key Helm Templates

templates/secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: {{ .Release.Name }}-secret
type: Opaque
data:
  SECRET_KEY: {{ .Values.secret.secretKey | b64enc }}
```

The b64enc filter base64-encodes the raw value passed via --set, so you pass the plain text secret and Helm handles the encoding that Kubernetes Secrets require.

templates/deployment.yaml (excerpt)

```
image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
envFrom:
  - configMapRef:
      name: {{ .Release.Name }}-config
env:
  - name: SECRET_KEY
    valueFrom:
      secretKeyRef:
        name: {{ .Release.Name }}-secret
        key: SECRET_KEY
```

Helm Commands

Install

```
helm install flask-app ./Phase\ 3 --set secret.secretKey=<your-plain-text-secret>
```

Upgrade (re-deploy after changes)

```
helm upgrade flask-app ./Phase\ 3 --set secret.secretKey=<your-plain-text-secret>
```

Check status

```
helm list
helm status flask-app
```

Uninstall

```
helm uninstall flask-app
```

Jenkins Setup

Jenkins Dockerfile (Phase 3/jenkins/Dockerfile)

```
FROM jenkins/jenkins:lts-jdk17

USER root

# Install Docker CLI and curl
RUN apt-get update && apt-get install -y docker.io curl

# Install kubectl for ARM64 (Apple Silicon)
RUN curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/arm64/kube
    && install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl \
    && rm kubectl

# Install Helm
RUN curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Allow jenkins user to use Docker
RUN usermod -aG docker jenkins

USER jenkins
```

The ARM64 kubectl download is required on Apple Silicon (M1/M2/M3) Macs. On Intel, change arm64 to amd64 in the curl URL. Helm is installed via its official script which auto-detects the architecture.

Jenkins docker-compose.yml (Phase 3/jenkins/docker-compose.yml)

```
services:
  jenkins:
    build: .
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
      - ./kubeconfig:/var/jenkins_home/.kube/config
    environment:
      - DOCKER_HOST=unix:///var/run/docker.sock
    entrypoint: >
      sh -c "chmod 666 /var/run/docker.sock && /usr/local/bin/jenkins.sh"

volumes:
  jenkins_home:
```

The Docker socket is bind-mounted so Jenkins can run docker build/push commands against the host Docker daemon. The entrypoint runs chmod 666 on the socket before starting Jenkins to fix the permission issue

described below. The kubeconfig is mounted so the Deploy stage can reach the Minikube cluster.

Starting Jenkins

```
cd "Phase 3/jenkins"
docker-compose up -d

# Get the initial admin password
docker exec jenkins-jenkins-1 cat /var/jenkins_home/secrets/initialAdminPassword

# Jenkins is at: http://localhost:8080
```

Jenkins First-Time Configuration

1. Unlock and install plugins

- Open http://localhost:8080 and paste the initial admin password
- Choose Install suggested plugins - this includes Git, Docker Pipeline, Credentials Binding
- Create your admin user and complete the setup wizard

2. Add Docker Hub credentials

- Manage Jenkins -> Credentials -> (global) -> Add Credentials
- Kind: Username with password
- ID: dockerhub-credentials (must match exactly - the Jenkinsfile references this ID)
- Username: your Docker Hub username
- Password: Docker Hub access token (not your account password)

Generate a Docker Hub access token at: hub.docker.com -> Account Settings -> Security -> Access Tokens

3. Create the pipeline job

- New Item -> name: flask-app -> Pipeline -> OK
- Definition: Pipeline script from SCM
- SCM: Git
- Repository URL: https://github.com/byakugen-sketch/devops-experts-project.git
- Branch Specifier: */main
- Script Path: Jenkinsfile
- Save -> Build Now

Jenkinsfile

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "uzumaki420/devops-experts-project"
        IMAGE_TAG  = "latest"
```

```
    }

    stages {
        stage('Checkout') {
            steps { checkout scm }
        }

        stage('Build') {
            steps {
                sh "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} ./Phase\ 1"
            }
        }

        stage('Test') {
            steps {
                sh "docker run --rm ${IMAGE_NAME}:${IMAGE_TAG} python -m pytest tests/ -v"
            }
        }

        stage('Push') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'dockerhub-credentials',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_PASS'
                )]) {
                    sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin'
                    sh "docker push ${IMAGE_NAME}:${IMAGE_TAG}"
                }
            }
        }

        stage('Deploy') {
            steps {
                sh "helm upgrade --install flask-app ./Phase\ 3 --set secret.secretKey=${IMAGE_TAG}"
            }
        }
    }

    post {
        success { echo 'Pipeline completed successfully' }
        failure  { echo 'Pipeline failed' }
    }
}
```

The pipeline runs Checkout -> Build -> Test -> Push -> Deploy sequentially. Push and Deploy only run if all previous stages pass. The Deploy stage uses helm upgrade --install which creates the release on first run and upgrades it on subsequent runs.

Pipeline Stage Summary

Stage	What it does
Checkout	Pulls the latest code from the GitHub repository
Build	Runs docker build from Phase 1 source, tags the image
Test	Runs pytest inside the freshly built container
Push	Logs in to Docker Hub and pushes the image
Deploy	Runs helm upgrade --install against the Minikube cluster

Issues Encountered & Fixes

[!] Issue: Jenkins defaulted to 'master' branch - repo uses 'main'

When creating the pipeline job, the Branch Specifier defaulted to */master. The repository uses main as the default branch, so the pipeline could not find the Jenkinsfile and failed at the Checkout stage with 'couldn't find remote ref refs/heads/master'.

[OK] Fix Applied

In the pipeline job configuration, change the Branch Specifier from */master to */main. This is a very common issue - GitHub changed the default branch name from master to main in 2020 and Jenkins was not updated to match.

[!] Issue: Docker socket permission denied inside Jenkins container

The Build stage failed with: 'Got permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock'. Even though the socket is mounted into the container, the jenkins user did not have permission to write to it.

[OK] Fix Applied

Added an entrypoint to docker-compose.yml that runs chmod 666 /var/run/docker.sock before starting Jenkins:

entrypoint: sh -c "chmod 666 /var/run/docker.sock && /usr/local/bin/jenkins.sh"

This makes the socket world-writable on every container start. The Dockerfile also adds the jenkins user to the docker group as a belt-and-suspenders measure.

[!] Issue: Kubernetes unreachable from Jenkins container - kubeconfig uses 127.0.0.1

The Deploy stage failed with: 'Unable to connect to the server: dial tcp 127.0.0.1:<port>: connect: connection refused'. The kubeconfig exported from Minikube uses 127.0.0.1 as the API server address. Inside the Jenkins container 127.0.0.1 refers to the container itself, not the host machine where Minikube is running.

[OK] Fix Applied

Copied the kubeconfig to Phase 3/jenkins/kubeconfig and replaced the server address:

FROM: server: https://127.0.0.1:<port>

TO: server: https://host.docker.internal:<port>

host.docker.internal is a special DNS name that Docker resolves to the host machine's IP from inside any container. The kubeconfig file is mounted into the container via the volume in docker-compose.yml.

[!] Issue: TLS certificate mismatch - certificate issued for localhost, not host.docker.internal

After fixing the server address, the Deploy stage failed with a TLS error: 'certificate is valid for localhost, not host.docker.internal'. Minikube issues its certificate for localhost/127.0.0.1. When the client connects via host.docker.internal the hostname does not match and TLS verification rejects it.

[OK] Fix Applied

In the kubeconfig, removed the certificate-authority-data field from the cluster entry and added insecure-skip-tls-verify: true:

```
cluster:
  insecure-skip-tls-verify: true
  server: https://host.docker.internal:<port>
```

This is acceptable for a local development cluster. It would NOT be acceptable in production where proper TLS verification is required.

[!] Issue: Docker Hub authentication failed - credentials rejected after password reset

The Push stage failed with: 'unauthorized: incorrect username or password'. A Docker Hub account password reset had invalidated the stored credentials in Jenkins.

[OK] Fix Applied

Generated a Docker Hub access token instead of using the account password:

hub.docker.com -> Account Settings -> Security -> Access Tokens -> New Access Token

Updated the Jenkins credential (ID: dockerhub-credentials) to use the token as the password field. Access tokens are the recommended approach regardless - they can be scoped and revoked without changing the account password.

Appendix - Quick Reference

Full End-to-End Rebuild Sequence

Run these commands in order from scratch to get the full pipeline running:

```
# 1. Start cluster
minikube start --driver=docker
minikube addons enable metrics-server

# 2. Start tunnel (keep open in separate terminal)
minikube tunnel

# 3. Build and push image
cd "Phase 1"
docker build -t uzumaki420/devops-experts-project:latest .
docker push uzumaki420/devops-experts-project:latest
cd ..

# 4. Create secret
python3 -c "import secrets, base64; print(base64.b64encode(secrets.token_hex(32).encode()).decode())"
cp "Phase 2/secret.yaml.example" "Phase 2/secret.yaml"
# Edit Phase 2/secret.yaml and fill in the generated value

# 5. Deploy to Kubernetes (Phase 2 - raw manifests)
kubectl apply -f "Phase 2/configmap.yaml"
kubectl apply -f "Phase 2/secret.yaml"
kubectl apply -f "Phase 2/deployment.yaml"
kubectl apply -f "Phase 2/service.yaml"
kubectl apply -f "Phase 2/hpa.yaml"
kubectl apply -f "Phase 2/cronjob-healthcheck.yaml"
kubectl apply -f "Phase 2/cronjob-cleanup.yaml"

# OR deploy via Helm (Phase 3)
helm install flask-app ./Phase\ 3 --set secret.secretKey=<your-secret>

# 6. Start Jenkins
cd "Phase 3/jenkins"
docker-compose up -d

# 7. Get admin password
docker exec jenkins-jenkins-1 cat /var/jenkins_home/secrets/initialAdminPassword
```

Useful kubectl Commands

```
kubectl get pods                        # list all pods
```

```
kubectl get pods -w                    # watch for changes
kubectl get services                   # list services with IPs
kubectl get hpa                        # autoscaler status
kubectl get cronjobs                   # list cron jobs
kubectl get jobs                       # completed job runs
kubectl describe pod <name>            # events, probes, env vars
kubectl logs <pod-name>                # pod output
kubectl logs -l app=flask-app          # all pods matching label
kubectl top pods                       # CPU and memory usage
kubectl delete pod <name>              # force pod restart
```

Useful Helm Commands

```
helm list                              # installed releases
helm status flask-app                  # release status
helm get values flask-app              # values in use
helm template flask-app ./Phase\ 3   # dry-run render templates
helm lint ./Phase\ 3                 # validate chart
helm uninstall flask-app               # remove everything
```

Useful Minikube Commands

```
minikube status                        # cluster status
minikube start --driver=docker         # start cluster
minikube stop                          # stop cluster
minikube tunnel                        # expose LoadBalancer IPs
minikube addons enable metrics-server  # enable metrics
minikube delete                        # destroy cluster
```

Endpoints

Endpoint	Method	Response
`/`	GET	`{"message": "Hello, Doron!"}`
`/health`	GET	`{"status": "healthy"}`
`/version`	GET	`{"version": "1.0.0"}`

Issues Summary

All issues encountered across all three phases, in order:

Phase	Issue	Fix
1	Home route returned plain text	Changed to jsonify(); updated test assertion
2	HPA	